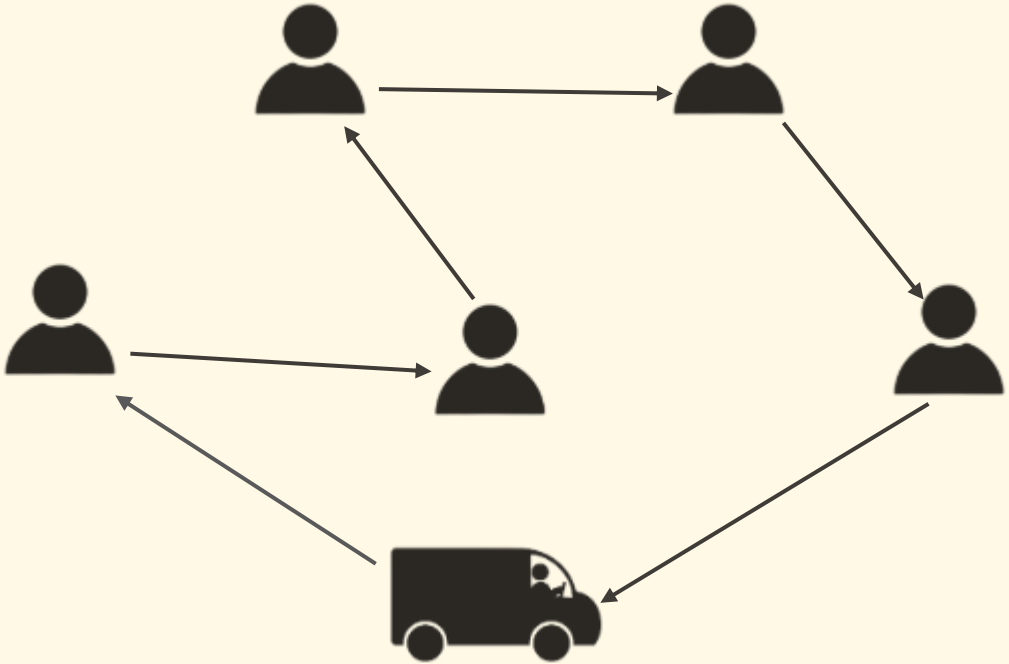
Windowed Encoding and Phase-Aware Deep Reinforcement Learning for Deterministic Time-Dependent TSP

STUDENT: Anuradha Dissanayake, 1747776
SUPERVISORS: Prof. Dr. Dr. Lars Schmidt-Thieme
Tim Dernedde

Table of contents

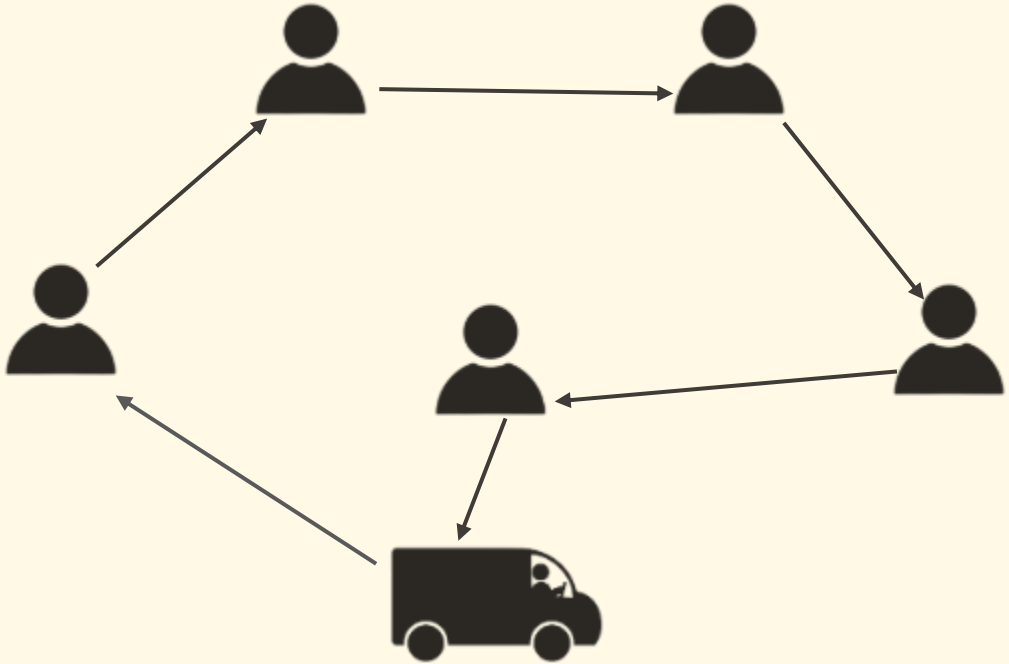
What is Traveling Salesman Problem?

Find the **shortest tour** that visits each customer **exactly once** and returns to the start



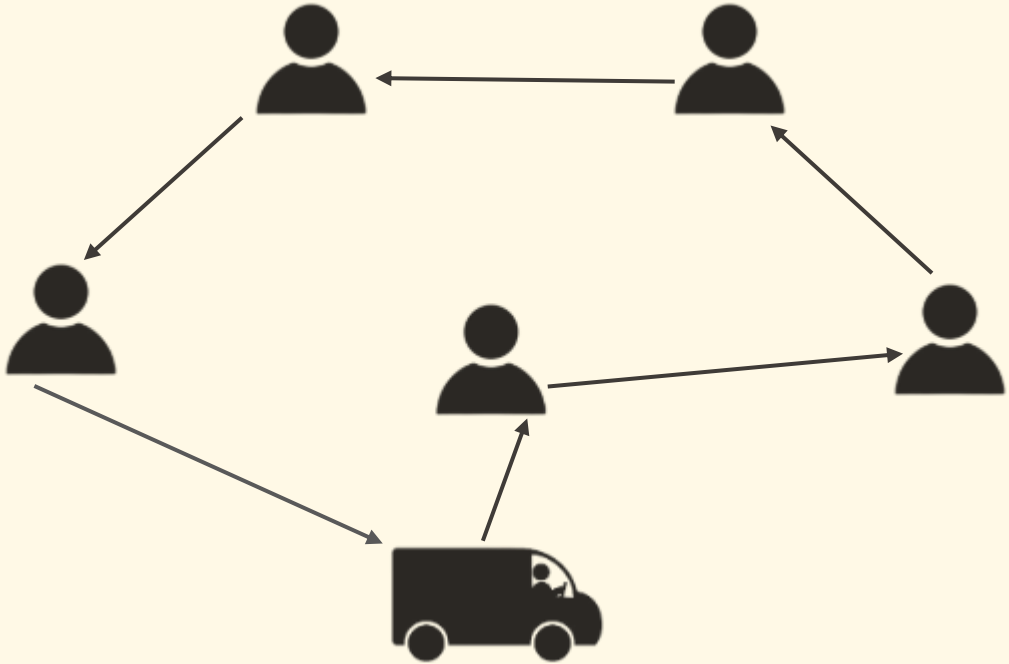
What is Traveling Salesman Problem?

Find the **shortest tour** that visits each customer **exactly once** and returns to the start



What is Traveling Salesman Problem?

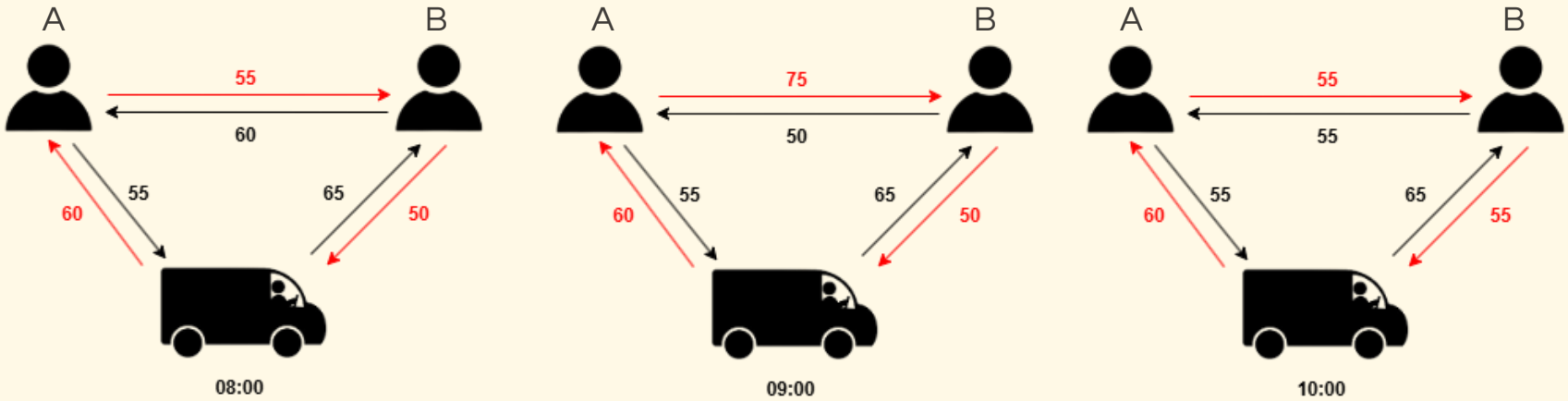
Find the **shortest tour** that visits each customer **exactly once** and returns to the start



Assumption: Costs between city pairs are fixed and do not vary over time.

Time-Dependent TSP (TDTSP)

Travel time between two locations can vary depending on the departure time due to factors like traffic congestion, rush hours, maintenance and others.



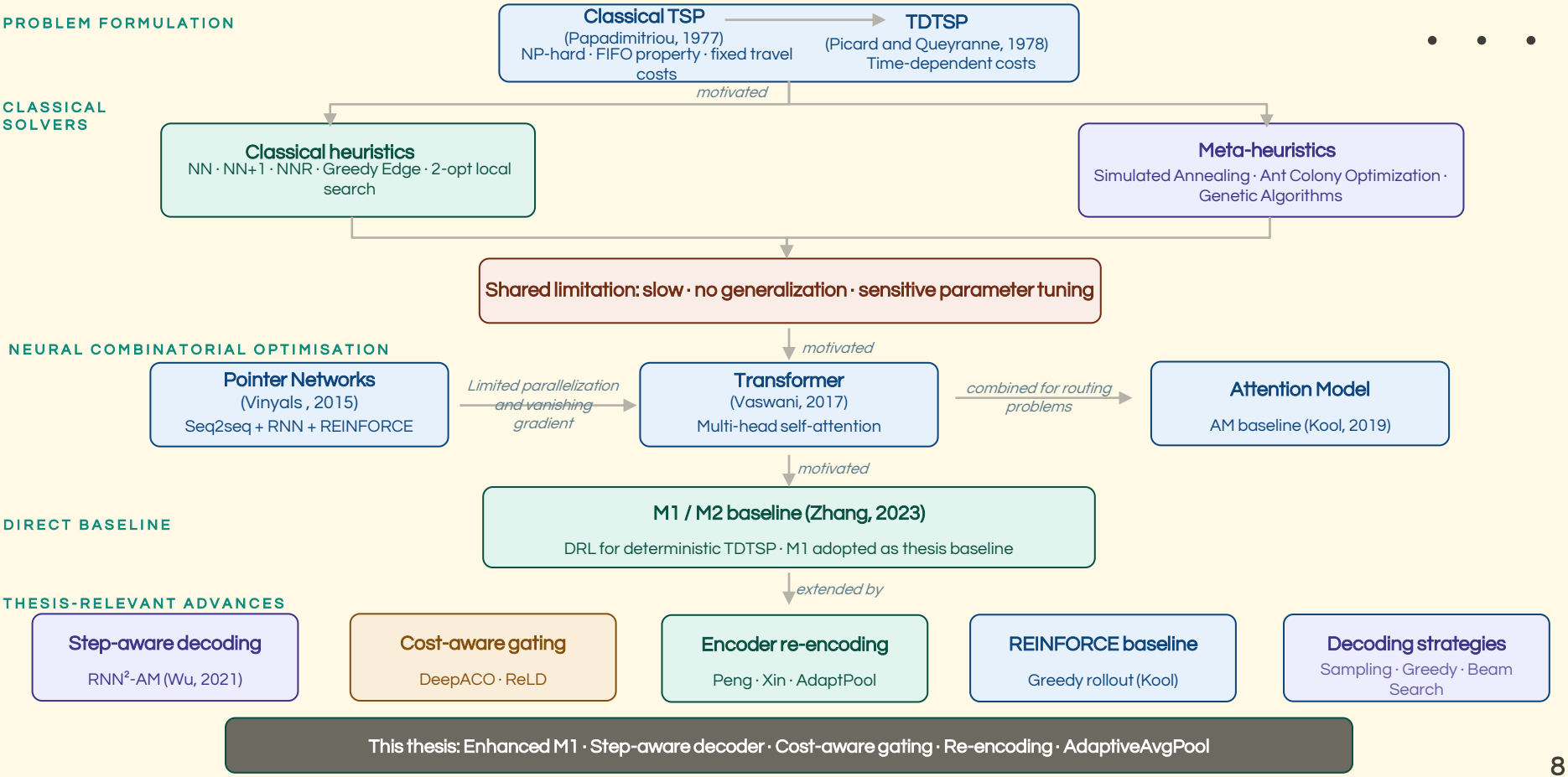
Assumption: First-In-First-Out (FIFO) property

Ex: If the driver departs at 08:00 from A to B, he arrives at 08:55. But if he departs at 08:30, he arrives at 08:40. That can't be possible. Departing later cannot result in arriving earlier.

Problem scope

- **The Problem**: We're solving the Time-Dependent Traveling Salesperson Problem (TDTSP), where travel times between locations change throughout the day (e.g., due to traffic).
- **Assumptions**:
  - We are primarily focused on the deterministic TDTSP : Travel times are known functions of departure time and not random variables (Stochastic TDTSP is outside the scope of this thesis and is left for future work).
  - We are working with a fixed customer set: all customers are known at tour start and do not change during the tour. (Dynamic customer pools, where customers appear or disappear mid-tour, are handled by M2 and not the focus of this thesis.)
  - FIFO property: departing later cannot result in arriving earlier. Guaranteed by real Beijing traffic data.
  - No waiting times at nodes.
- **Baseline. Zhang et al. (2023)** propose two approaches inspired by an attention-based encoder–decoder architecture. We use this model as our baseline.
- Decisions are made sequentially (the next node is chosen after each arrival).
- Dataset: **Real-world Beijing traffic data** (Zhang et al., 2023) and it contains 100 city locations, 12 two-hour time bins, cubic spline travel time representation.

Related Work



Baseline Approach and Models

The baseline paper (*Solving Dynamic Traveling Salesman Problems with Deep Reinforcement Learning)* by *Zhang et al. (2023),* which inspired this thesis, builds upon this attention-based approach with a few modifications.

M1 Model (Encode Once, Decode Online)

Encoder

- Processes static customer information and entire day's traffic patterns once to generate embeddings
- Produces node and overall graph embeddings using layers of self-attention and normalization.

Decoder

- Sequentially selects the next node at each step.
- Only updates the current state based on travel time and visited mask (**does not re-encode** after the initial pass).

Best For: Fixed customer sets. M1 outperforms M2 by >5% on fixed customer sets

M2 Model (Re-encode Every Step)

Encoder

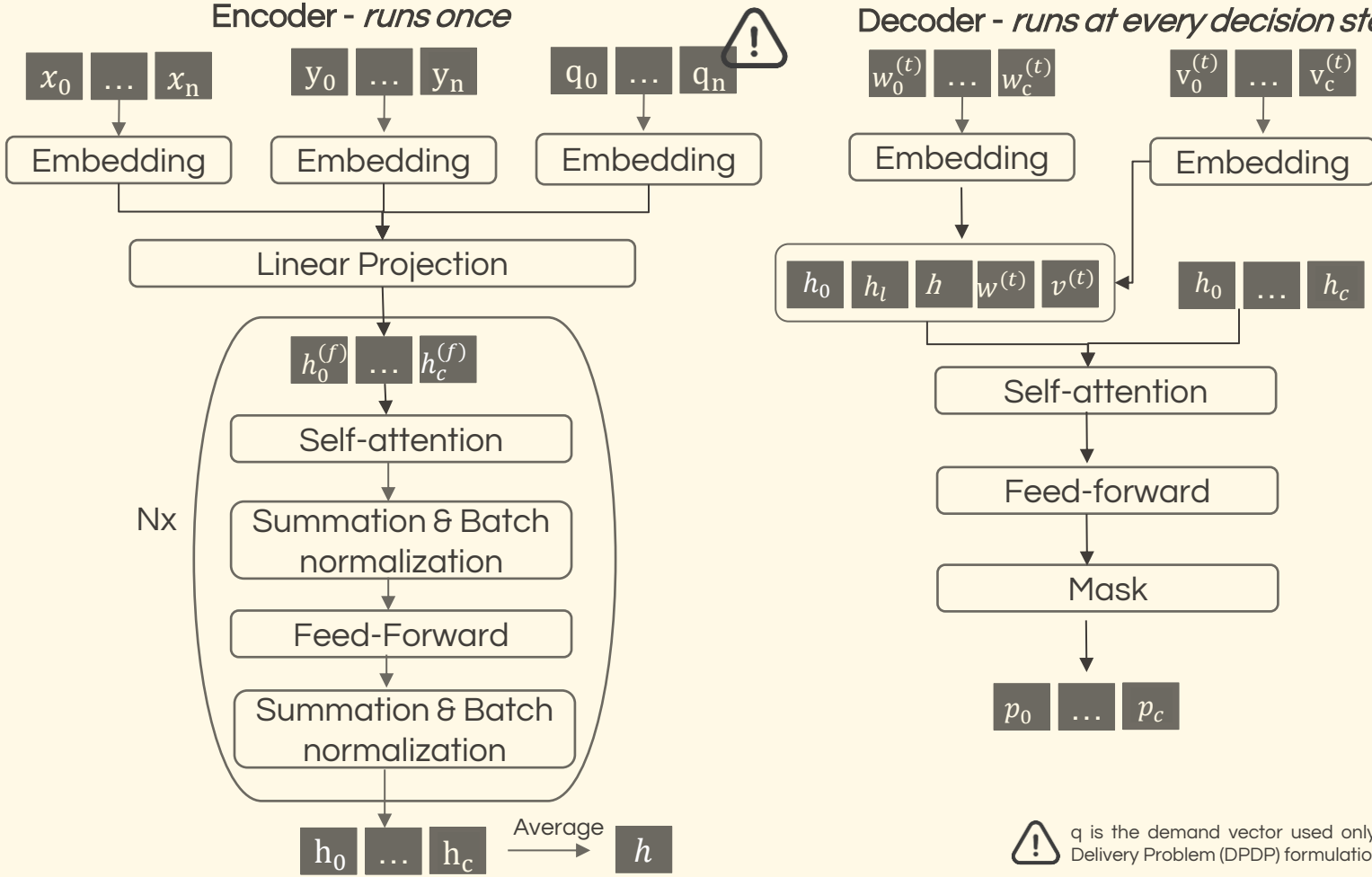
- Combines Node2Vec (node embedding) and Traffic2Vec (traffic embedding) for richer representations.
- At every step, re-encodes current node and traffic state before decoding the next move.

Decoder

- Utilizes updated embeddings at every decision point, allowing adaptation to dynamic changes (customers added/removed).

Best For: Dynamic customer pools.

M1 Model - Current



q is the demand vector used only for the Dynamic Pickup and Delivery Problem (DPDP) formulation and is not used in TDTSP

Identified Gaps and Our Solutions

While the M1 model is effective, we've identified three key areas for improvement. Our proposed enhancements directly address each of these gaps.

The Gap	Our Proposed Solution
1. Tour-Progress-Blind Decoder The policy's decoder lacks memory beyond the last visited node (no tour-progress awareness).	**Step-MLP** Introduce a lightweight MLP that takes step features and provides a contextual nudge to the decoder query.
2. Missing Cost Bias The model learns a policy but ignores classical heuristic priors.	**Cost-Aware Gating (Soft Bias)** Add heuristic logits (NN, NN+1, NNR) as a learnable soft bias to the model's raw attention logits.
3. Time-Insensitive Encoder The model encodes the full day's traffic upfront, even as most of it becomes irrelevant during the tour.	**Time Slicing with Safe Refresh** Feed only a forward window of traffic data to the encoder with safe-refresh re-encoding, making it more efficient.

Problem Formulation: TDTSP

Formal Definition

G Complete directed graph $G = (V, E)$

$V = \{0, 1, 2, \ldots, n\}$ · $E = V \times V$ (all directed arcs)

d Time-dependent travel time

$d_{ij}(t)$ = travel time from i → j departing at time t

π Tour $\pi = (\pi_0, \pi_1, \ldots, \pi_n, \pi_{n+1})$

$\pi_0 = \pi_{n+1} = 0$ (depot) · Visit each customer exactly once

Objective Function

$$\min_{\pi} \sum_{k=0}^{n} d_{\pi_k, \pi_{k+1}}(t_{\pi_k})$$

k = 0, 1, …, n (sum over all consecutive tour stops)

Arrival Time Recursion

$$t_{\pi_0} = 0$$
$$t_{\pi_{k+1}} = t_{\pi_k} + d_{\pi_k, \pi_{k+1}}(t_{\pi_k}), \quad k = 0, 1, \ldots, n$$

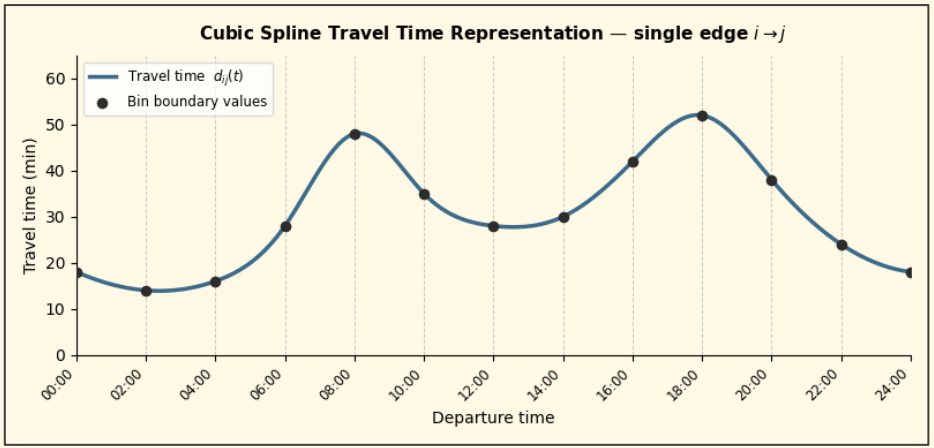
Travel Time Representation: Cubic Splines

24-hour day split into T = 12 bins of 2 hours each

Four coefficients (c^0, c^1, c^2, c^3) fitted per edge per bin:

$$d_{ij}(t) = c^0_{ij,b} + c^1_{ij,b} z + c^2_{ij,b} z^2 + c^3_{ij,b} z^3$$

b = ⌊t·T⌋ (current bin index, 0-11)
z = (t·T − ⌊t·T⌋) / T ∈ [0, 1/T) (fractional position within the current 2-hour bin)



MDP Framework for the TDTSP

We formulate the TDTSP as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ) and solve it using a deep reinforcement learning agent. The framework consists of four main components.

State Space

A state s_k at decision step k comprises all information needed to make the next routing decision (Static and Dynamic attributes)

Action Space

At each step k, the action a_k is the index of the next node to visit. The action space is state-dependent, containing only unvisited nodes

Reward

The reward is the negative travel time incurred by the transition, converting the cost minimisation objective into reward maximisation

Agent / Policy

The policy $\pi_\theta(a_k \mid s_k)$ is parameterised by the M1 encoder-decoder architecture and outputs a probability distribution over the action space $A(s_k)$.

Policy Factorisation

$$p_\theta(\pi \mid s) = \prod_{k=1}^{N} p_\theta(\pi_k \mid s, \pi_1, \ldots, \pi_{k-1})$$

where $\pi = [\pi_1, \pi_2, \ldots, \pi_N]$ is the complete tour sequence, N is the number of nodes (graph size + 1, including depot), θ represents the learnable parameters, and each $p_\theta(\pi_k \mid \cdot)$ is the probability distribution over unvisited nodes at step k.

Learning Objective

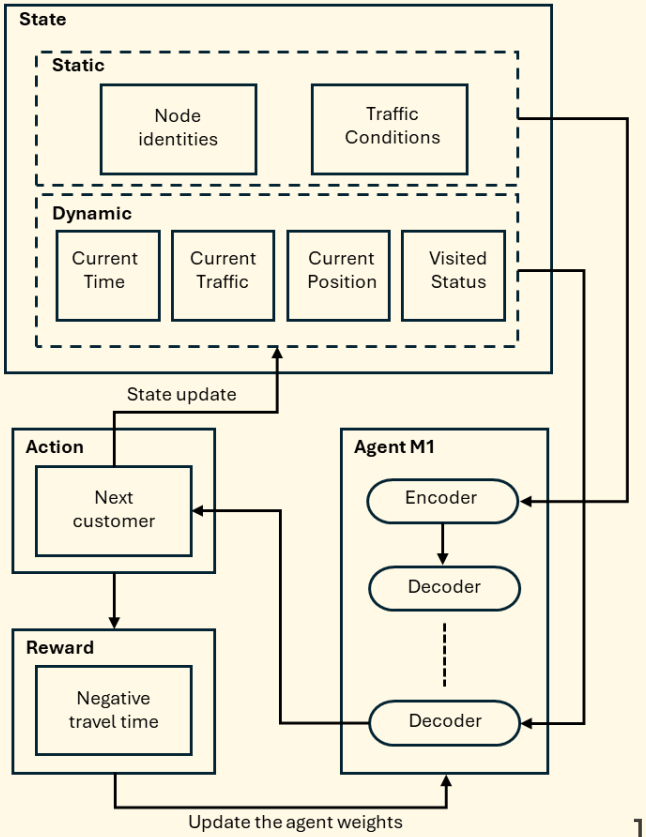
To find policy parameters θ^* that maximise the expected cumulative return:

$$\theta^* = \arg\max_\theta \; \mathbb{E}_{s\sim\mathbb{D}} \, \mathbb{E}_{\pi\sim\mathbb{p}_\theta(\cdot|\mathbb{S})}[G(\pi, s)]$$

where $\mathcal{D}$ is the distribution of problem instances and $G(\pi, s)$is the return (negative total travel time) for tour π on instance s.

Train: $a_t \sim \pi_\theta(\cdot \mid s_t)$(sampling)

Eval: $a_t = \arg\max_{a\in A(s_t)} \pi_\theta(a \mid s_t)$(greedy) / beam



Step-MLP and Decoder Variants

The decoder has no awareness of how far through the tour it has progressed. We solve this by injecting a step-aware signal into the decoder at different positions. We tested four mechanisms to find where this injection is most effective.

MECHANISM 1

Step-MLP (context nudge)

MLP processes step features (step ratio, last-3 nodes, time, depot dist, …) and adds a vector Δq to the decoder query. A three-layer MLP processes the step feature vector $f_{\text{step}} \in \mathbb{R}^{132}$

$$h_1 = \text{ReLU}(W_1 f_{\text{step}} + b_1),\ W_1 \in \mathbb{R}^{64\times 132},\ h_1 \in \mathbb{R}^{64}$$
$$h_2 = \text{ReLU}(W_2 h_1 + b_2),\ W_2 \in \mathbb{R}^{64\times 64},\ h_2 \in \mathbb{R}^{64}$$
$$\Delta q = W_3 h_2 + b_3, ; W_3 \in \mathbb{R}^{128\times 64}, ; \Delta q \in \mathbb{R}^{128}$$
$$q_{\text{final}} = q_{\text{base}} + \Delta q$$

MECHANISM 2

Pre-attention MLP (Option C)

A two-layer feed-forward network nonlinearly refines the query q_t before the attention computation:

$$h_1 = \text{ReLU}(W_1 q_{\text{old}} + b_1),\quad W_1 \in \mathbb{R}^{512\times 128},\quad h_1 \in \mathbb{R}^{512}$$
$$\text{refined}_q = W_2 h_1 + b_2,\quad W_2 \in \mathbb{R}^{128\times 512},\quad \text{refined}_q \in \mathbb{R}^{128}$$

The refined output is added back as a residual:

$$q_{\text{new}} = q_{\text{old}} + \text{refined}_q,\quad q_{\text{old}}, q_{\text{new}} \in \mathbb{R}^{128}$$

MECHANISM 3

Post-attention MLP (Option A)

In the baseline decoder, only a single linear projection is used after the MHA for the glimpse(stage 1). We propose a post-attention MLP that refines the glimpse $g_t \in \mathbb{R}^{128}$ after the MHA but before the final logit computation:

$$h = \text{ReLU}(W_1 g_t + b_1), W_1 \in \mathbb{R}^{512\times 128}$$
$$g_{\text{refined}} = W_2 h + b_2, W_2 \in \mathbb{R}^{128\times 512}$$

The refined glimpse replaces the original (no residual connection): $g_{\text{final}} = g_{\text{refined}}$. Final logits are then:

$$\ell_i = \frac{g_{\text{final}}^T k_i^{\text{logit}}}{\sqrt{d}}$$

MECHANISM 4

Temperature MLP (Temp-MLP)

Lightweight MLP scales attention logits with a learned temperature τ_step.

$$h = \text{ReLU}(W_1 f_{\text{step}} + b_1), W_1 \in \mathbb{R}^{64\times 132}, h \in \mathbb{R}^{64}$$
$$\tau_{\text{raw}} = \text{Sigmoid}(W_2 h + b_2), W_2 \in \mathbb{R}^{1\times 64}, \tau_{\text{raw}} \in [0,1]$$
$$\tau_{\text{step}} = 0.5 + 2.0 \cdot \tau_{\text{raw}} \in [0.5, 2.5]$$

log-probabilities are computed as:

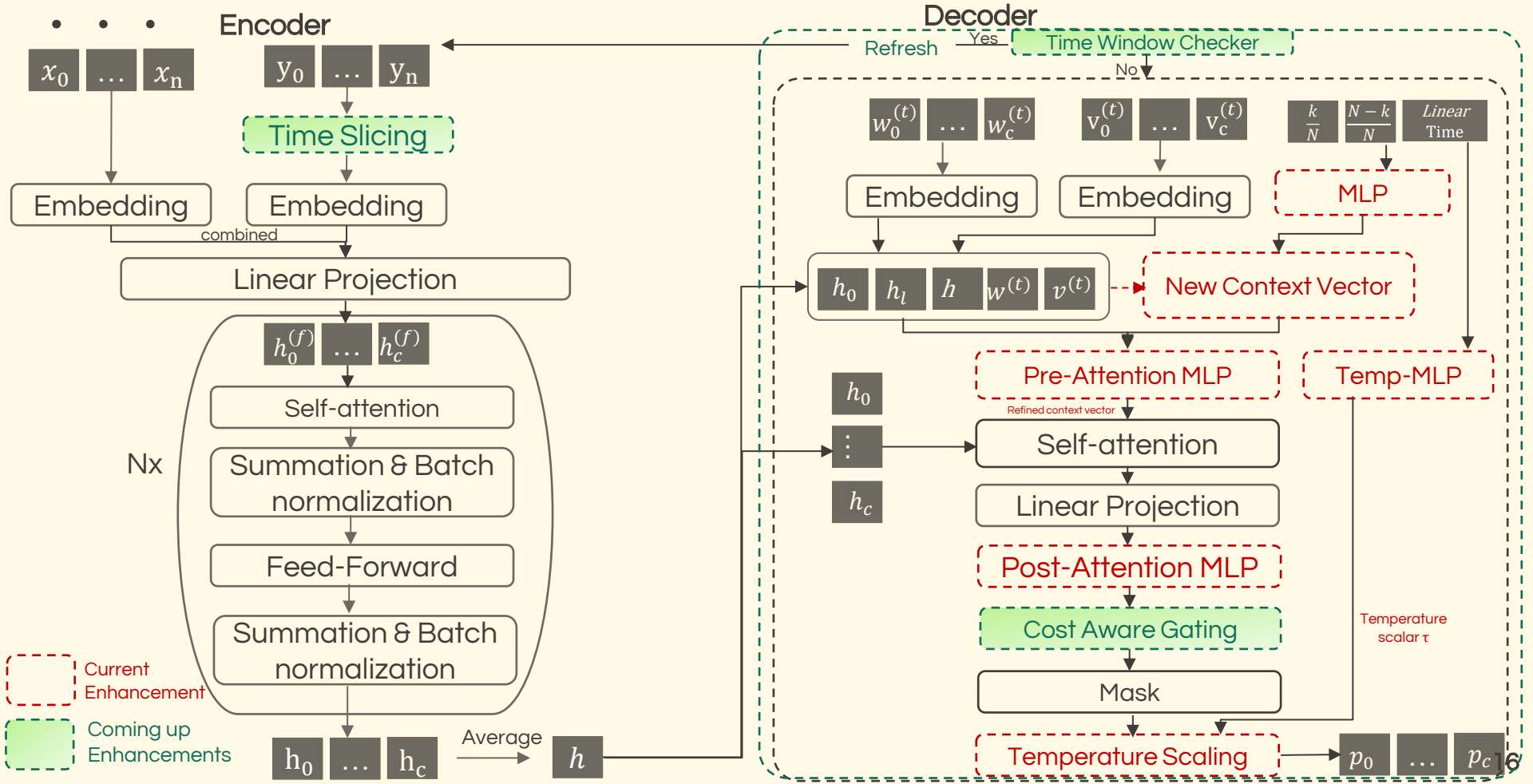
$$\log(p_j) = \log\left(\text{softmax}\left(\frac{\ell}{T\cdot\tau_{\text{step}}}\right)_j\right),\ \textit{where T global hyperparameter(default 1)}$$

Step Features

Feature	Dim.	Description	Formula
Step ratio	1	Progress through tour	$r = k/N$
Last-3 nodes	3	Recent visit pattern	$[1_{k-1},\ 1_{k-2},\ 1_{k-3}]$
Visited mean	128	Spatial tour context	$\bar{h}_{\text{vis}} = \frac{1}{\sum_t v_t} \sum_{t=0}^{n} v_t h_t$
Unvisited mean	128	Remaining option context	$\bar{h}_{\text{unvis}} = \frac{1}{\sum_t (1-v_t)} \sum_{t=0}^{n} (1-v_t) h_t$
Sin/Cos time	2	Cyclic time encoding	$[\sin(2\pi t),\ \cos(2\pi t)]$
Linear time	1	Direct time encoding	$t = \text{state.lengths}$
Depot distance	1	Distance back to depot	$d_{i_t,0}(t)$
Tour length	1	Cumulative travel time	L_t (normalised by $0.1 \cdot N$)
Mean dist to unvisited	1	Average cost to remaining	$\bar{d}_{\text{unvis}} = \frac{1}{\sum_j (1-v_j)} \sum_{j:\, v_j=0} d_{i_t,j}(t)$

- Individual features (step ratio, linear time, depot distance, tour length, mean distance to unvisited nodes, and last 3 visited nodes) were first evaluated independently.
- Five predefined feature combinations were then compared.
- The lightweight *v2_light* **(step ratio + linear time + depot dist + tour length + mean dist to unvisited)** configuration achieved the best performance at C= 19.
- Leave-one-out analysis identified linear time as the most influential feature.

M1 Model – with Enhancement



Cost-Aware Gating and Heuristic Blending

Inject classical routing knowledge as a soft bias on attention logits.

HEURISTIC 1

Linear travel time (NN)

This heuristic selects nodes with the shortest travel time from the current node

$$h_i^{\text{lin}} = -d_{i_t,i}$$

HEURISTIC 2

NN+1 lookahead

Selects nodes based not only on the immediate cost but also on how good the next move will be.

immidiate cost: $c_i = d_{i,i(t)}$

$Lookahead\ cost: c_i^{\text{next}} = \min_{j \in U_t \setminus \{i\}} d_{i,j}(t + c_i)$

$$\text{score}_i = c_i + \lambda_{\text{lookahead}} \cdot c_i^{\text{next}}$$
$$h_i = -\text{score}_i\,,\ \lambda_{\text{lookahead}} = 0.5$$

HEURISTIC 3

NN-Random (top-k)

Assigns probability-based scores to the top-k (k = 3) candidate nodes. Candidates are sorted by travel time (i_1 has shortest) and assigned probabilities:

$$h_i^{\text{NNR}} = \begin{cases} \log(0.80) \approx -0.22 & \text{if } i = i_1 \\ \log(0.15) \approx -1.90 & \text{if } i = i_2 \\ \log(0.05) \approx -3.00 & \text{if } i = i_3 \\ -10.0 & \text{otherwise} \end{cases}$$

The heuristic scores are blended with the attention logits ℓ_i:

$$\ell_i^{\text{blended}} = \ell_i + \lambda_{\text{heur}} \cdot h_i(s_t)$$

where $\lambda_{\text{heur}} \in [0,2.0]$ is a learnable parameter, initialized to $\lambda_{\text{heur}} = 1.0$

The heuristic logits can be transformed nonlinearly before blending, either using:

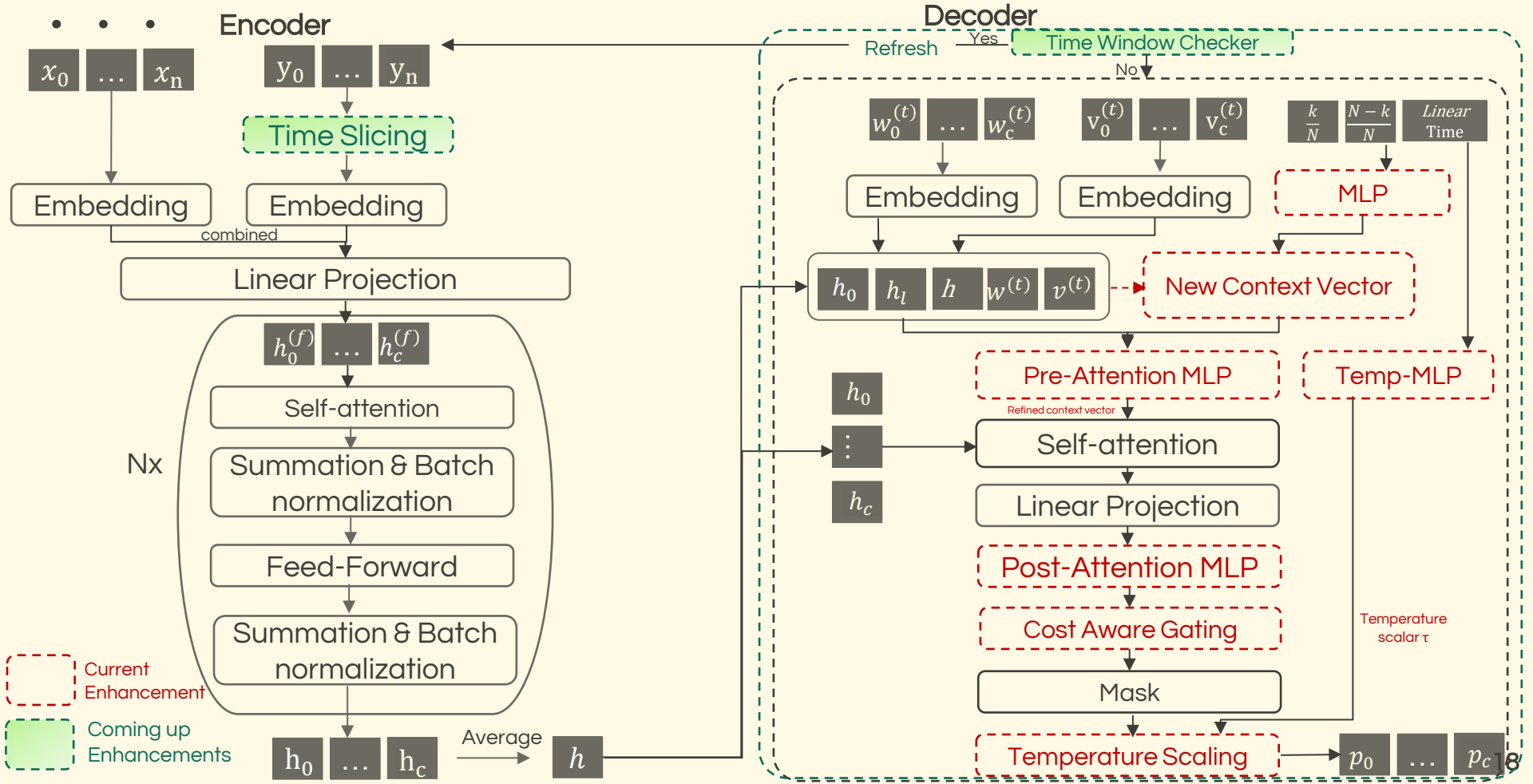
Piecewise linear

$$h_i^{\text{transformed}} = \text{MLP}_{\text{transform}}(h_i)$$

Exponential

$$h_i^{\text{transformed}} = \alpha \cdot \exp(h_i + \beta)\,, \qquad \alpha, \beta \in \mathbb{R}$$

M1 Model – with Enhancement



Time-Sliced Traffic Encoding and Safe Refresh

Why slice?

The baseline M1 model feeds the full-day traffic profile for every arc (i,j) into the encoder. However, real-world routing applications such as logistics and food delivery operate within limited time horizons, making much of this information irrelevant.

Therefore, we use a ***forward-looking time window*** from the current decision time, reducing input size and computational complexity while maintaining performance..

Our approach has three main components

Slicing the traffic tensor to a relevant forward window

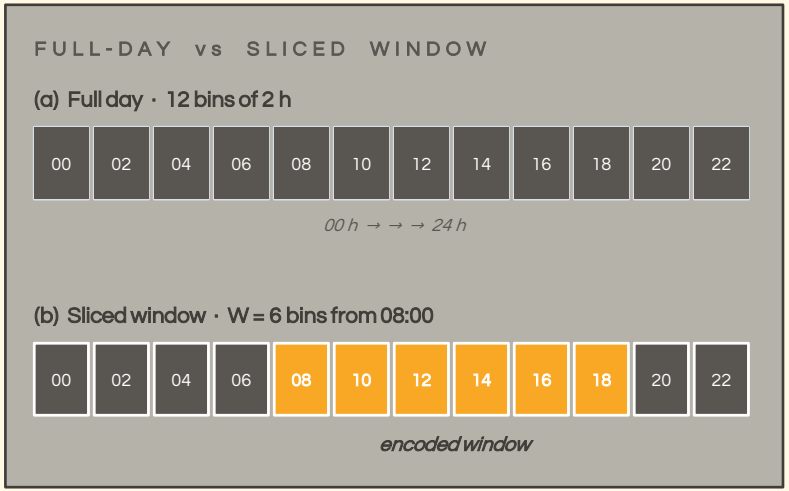
Reshape traffic to (n_c, n_c, T) and take W forward bins.

Embed

Embed the windowed tensor instead of the full day.

Refresh

Monitor tour time and re-encode when the window goes stale.



Four strategies to keep the encoded window fresh

The encoder must be re-run when the current decision time approaches or exceeds the encoded window, otherwise the model uses stale traffic.

1

One-Time Refresh Check

If the decoder's current bin exceeds the window end bin, the encoder re-runs with the latest values before the next decoding step, setting the new start bin to the current bin.

2

Periodic Refresh

A refresh is triggered on a fixed schedule regardless of window boundaries. At each decoding step, it computes time since refresh = current time – last refresh time and triggers a refresh if time since refresh ≥ refresh interval. This mitigates the risk of using stale embeddings.

3

k-Moves Lookahead

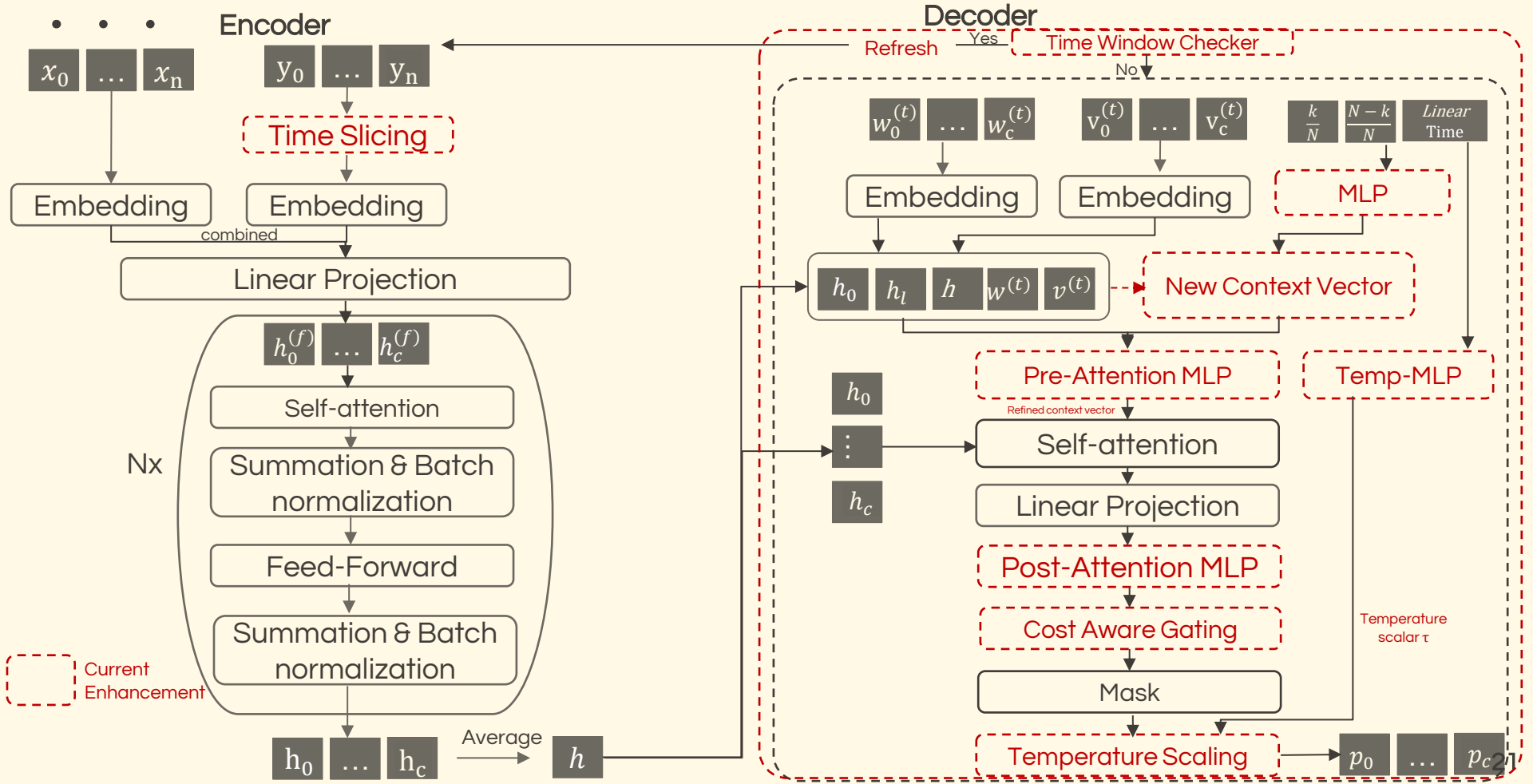
The encoder is refreshed before the boundary is reached by simulating k moves ahead. The future bin is estimated either by adding k time bins to the current time, or by simulating the next k moves via greedy decoding or nearest-neighbour methods. In this thesis, we adopt the approach of adding k time bins to the current time, as it is simpler and computationally more efficient than simulating future moves. A refresh is triggered if the estimated future bin ≥ the current window end bin.

4

Combined

A combination of all three strategies; the encoder is refreshed whenever any of their criteria is satisfied

M1 Model – with Enhancement



Policy Gradient: REINFORCE Algorithm

- TDTSP has no labelled tour data, reinforcement learning is used to train the decoder. The objective is to learn a policy π_θ that minimises the expected tour cost:

$$J(\theta) = \mathbb{E}_{s \sim \mathbb{D}_{\text{train}}}\left[C(\pi_\theta(s))\right]$$

- For each instance, the model outputs a tour cost C and the log-likelihood of the sampled tour. The tour cost is:

$$C_k = \sum_{(i,j) \in \text{tour}} d_{ij}\left(t_{\text{arrival at } i}\right)$$

- The log-likelihood of the tour is:

$$\log p_\theta(\pi \mid x) = \sum_t \log p_\theta(a_t \mid s, a_{<t})$$

- Using the REINFORCE policy gradient estimator, the gradient of J(θ) is approximated by:

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\left(C(\pi_\theta(s)) - b(s)\right) \nabla_\theta \log p_\theta(\pi_\theta(s) \mid s)\right]$$

where b(s) is the baseline cost.

- For a mini-batch of size B, the batch-wise REINFORCE loss is:

$$L_{\text{REINFORCE}} = \frac{1}{B} \sum_{i=1}^{B} (C_i - b_i) \log p_\theta(\pi_i \mid x_i)$$

- After computing the loss, backpropagation is performed, followed by ℓ2-norm gradient clipping with threshold τ :

$$g = \nabla_\theta L_{\text{REINFORCE}} \quad , \quad \tilde{g} = \text{clip}_{\ell_2}(g, \tau) = \frac{g}{\max\left(1, \|g\|_2 / \tau\right)}$$

- Parameters are then updated using the Adam optimiser with learning rate α:

$$\theta \leftarrow \theta - \alpha \tilde{g}$$

Baseline Strategies

Three baseline strategies are implemented, the user can choose any option according to their requirements, each with different trade-offs.

Exponential Moving Average Baseline

- Maintains a moving average of batch costs, exponentially weighted towards recent batches.
- If a batch b consists of B instances with batch cost:

$$\widetilde{C_b} = \frac{1}{B}\sum_{k=1}^{B} C_{b,k}$$

- The baseline is updated as:

$$v_b = \begin{cases} \tilde{C}_0 & \text{if } b = 0 \\ \beta \cdot v_{b-1} + (1-\beta)\cdot \tilde{C}_b & \text{if } b > 0 \end{cases}$$

where β ∈ [0, 1] is the scaling factor (default : 0.8).

- The EMA is a scalar value (not a model) and is updated at every batch.

Rollout Baseline

- Maintain two models:
  - **Training model** (updated during training)
  - **Baseline model** (frozen copy)
- Initialize the baseline as a copy of the training model.
- After each epoch:
  - Evaluate both models on the validation set.
  - Compute the cost difference for each instance.
  - Perform a one-sided paired t-test.
- If the training model significantly outperforms the baseline ($p < 0.05$):
  - Update the baseline with the current model.
- Otherwise:
  - Keep the existing baseline.
- *Key idea: The baseline is updated only when the current model achieves statistically significant improvement.*

Warmup Baseline

- Early in training, the rollout baseline is unreliable because model weights are randomly initialized.
- Use an **Exponential Moving Average (EMA)** baseline during the first K_{WU}(number of warmups) epochs.
- Gradually transition from EMA to the rollout baseline using a blend factor:

$$\gamma(e) = \frac{\min(e+1, K_{WU})}{K_{WU}}$$

- Compute the blended baseline as:

$$b_i^{blended} = \gamma(e)\, b_i^{rollout} + \big(1-\gamma(e)\big)\, b_i^{EMA}$$

- As training progresses:
  - $\gamma(e)$increases from 0 to 1.
  - The baseline smoothly shifts from EMA to rollout.
  - Once $e \geq K_{\mathrm{WU}}$, the rollout baseline is used exclusively.
- *Key idea: EMA provides a stable baseline during early training, while the rollout baseline takes over as the model becomes more reliable.*

Graph-Size-Independent Architecture

- The baseline M1 model (Zhang et al., 2023) is trained for a fixed customer set size (C = 4, 9 and 19), requiring retraining whenever the graph size changes. This limitation motivates the development of a graph-size-independent architecture for real-world routing applications.
- Graph-size dependency mainly affects inside the decoder and specially two main dynamic components inside the context vector: visited state and traffic embedding.

Visited State Projection

At each decoding step, the model needs to know which nodes have already been visited. It builds a binary vector v(t) (0 : not yet visited, 1 : node already visited)

Trained on $n = 19$: $v(t) \in \mathbb{R}^{20}$, $W_v \in \mathbb{R}^{128\times 20}$

Linear projection layer: (128*20) . (20*1) = (128*1) ✓

Evaluated on n = 29: $v(t) \in \mathbb{R}^{30}$, $W_v \in \mathbb{R}^{128\times 20}$

Linear projection layer: (128*20) . (30*1) ✗ **crash**.

Traffic Embedding Projection

At each decoding step, decoder needs to know current travel time between every pair of nodes, not just from the current node. This requires first generating all pairs, then looking up their travel times, and finally projecting to 128 dimensions.

In pairwise combination creation process, baseline model saves the list in training using the input size and it is fixed.

$$xx = [\underbrace{0,\dots,0}_{20},\underbrace{1,\dots,1}_{20},\ \dots,\ \underbrace{19,\dots,19}_{20}] \in \mathbb{R}^{400}$$

$$yy = [\underbrace{0,\dots,19}_{20},\underbrace{0,\dots,19}_{20},\ \dots,\ \underbrace{0,\dots,19}_{20}] \in \mathbb{R}^{400}$$

Trained on n = 19: $w(t) \in \mathbb{R}^{400}$, $W_w \in \mathbb{R}^{128\times 400}$

Linear projection layer: (128*400) . (400*1) = (128*1) ✓

Evaluated on n = 29: $w(t) \in \mathbb{R}^{900}$, $W_w \in \mathbb{R}^{128\times 400}$

Linear projection layer: (128*400) . (900*1) ✗ **crash**.

Solution: AdaptiveAvgPool1d(128)

Fix 1: Dynamic _generate_xx_yy() : indices generated at runtime from actual graph size (replaces static self.xx, self.yy)

Fix 2 : Contains two layers in sequence.

Step 1 : **AdaptiveAvgPool1d(128)** : This layer has no learnable weights. It's a pure mathematical operation(average) that maps any input of length L to exactly 128 values($\mathbb{R}^{128\times 1}$).

Step 2: **nn.Linear(128, 128) :** Weight matrixes ($W_v \in \mathbb{R}^{128\times 128}$) and linear projection (Fixed weight matrix, always 128×128 regardless of graph size)

Experimental Setup

All experiments were executed on the university HPC cluster (STUD partition). Each experiment was allocated 4 CPU cores and a single NVIDIA GeForce RTX 2070 GPU with 8GB of memory.

Problem & Dataset

Real-world Beijing dataset (Zhang et al., 2023)

100	10,000	12
locations (incl. depot)	directed city pairs (i, j)	2-hour time bins / arc

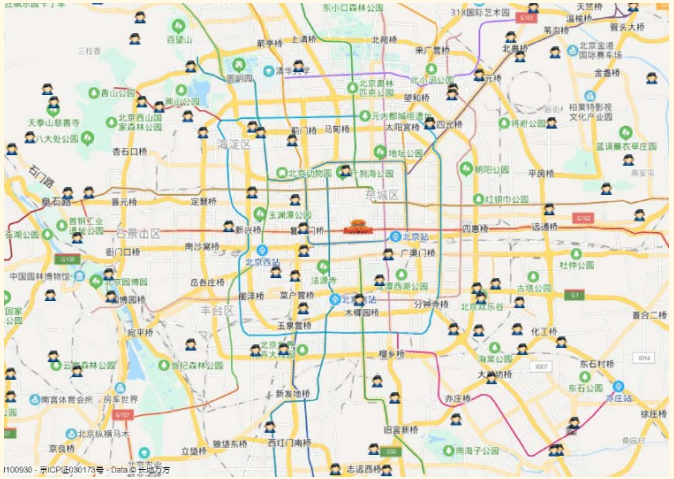
Preprocessing: normalised travel times smoothed via cubic spline interpolation

Node encoding: one-hot vectors (dim 100). Model uses time-dependent travel times, not Euclidean distances.

Customer Sets (C)

Random subsets of size C + 1 drawn from the 100 cities

C = 19	C = 20	C = 29	C = 49
Main experiments & comparisons	Hyperparameter optimisation	Scalability + graph dependency	Larger-scale scalability



Location map of the benchmark dataset

Dataset	Size	Sampling	Purpose
Training	128k	Random, regenerated each epoch	Train model and prevent memorization
Validation	1k	Fixed	Monitor performance and select checkpoints
Baseline Validation	1k	Regenerated after each accepted update	Compare against rollout baseline for t-test updates
Test	1k	Fixed across all runs	Final evaluation and fair comparison

Step-MLP - Feature Preset Comparison

Individual features were first evaluated independently. **None outperformed the baseline** (**574.7 min**), with results ranging from **577.9 to 581.5 min**. Among the individual features, **linear time** and **mean distance to unvisited nodes** achieved the best performance.

Feature preset comparison (C = 19, 50 epochs)

Feature Set	Baseline	v1 light (4)	minimal (129)	v1 (132)	v2 light (linear, 5)
Mean ± SD (min)	574.73 ± 10.78	580.00 ± 3.42	575.60 ± 8.00	574.95 ± 7.36	**573.80 ± 7.22**

- Best mean cost: **v2_light (step ratio + linear time + depot dist + tour length + mean dist to unvisited)** (573.80 min)
- Improvement over baseline: 0.93 min (not statistically significant)
- Lower variance than baseline:v2_light: ±7.22 vs Baseline: ±10.78
- Indicates more stable training across seeds
- Larger feature sets (129-132 dimensions) provided no consistent benefit
- Additional embedding-based features may introduce noise at this problem size

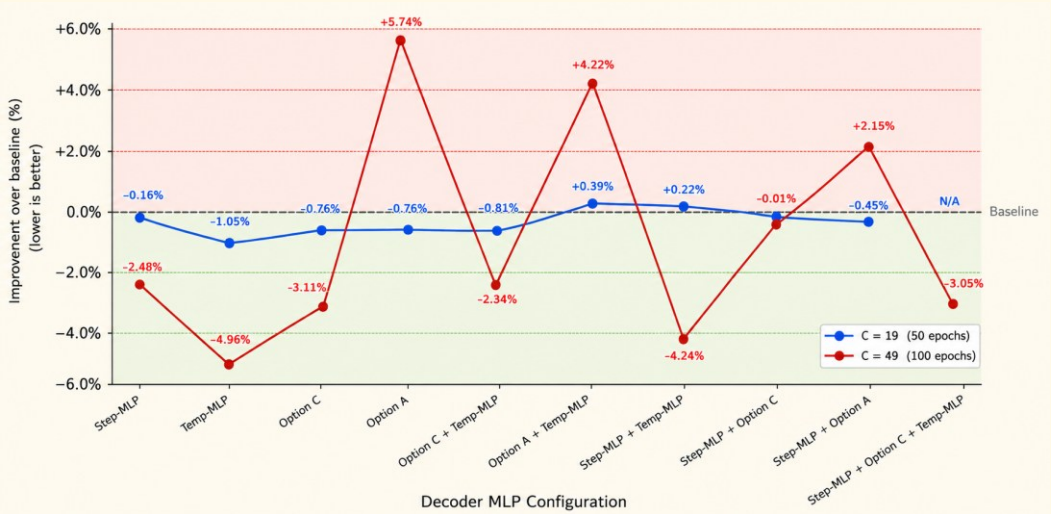
Leave-One-Out Ablation of v2_light

A leave-one-out ablation study was performed on the five features of **v2_light**.

Removed Feature	Mean Cost (min)	Δ vs. Full v2_light
None (full v2_light)	573.80	-
linear_time	581.34	+7.49
mean_dist_unvisited	578.80	+4.95
step_ratio	577.96	+4.11
tour_length	577.12	+3.27
depot_distance	574.55	+0.70

- Linear time was the most critical feature, with its removal increasing the cost by **7.5 min**.
- Replacing linear time with sin-cos encoding also degraded performance (**578.2 vs. 573.8 min**).
- This suggests that linear time is better suited to a single operational day than periodic encodings.
- Based on its lowest mean cost and stable training behaviour, **v2_light with linear time encoding** was selected for all subsequent experiments.

Decoder Variants



BEST CONFIGURATION

Temp-MLP standalone

C = 19

-1.05% 568.68 ± 2.12min

C = 49

-4.96% 1041.57 ± 19.61 min

WORST PERFORMER

Option A (Degrades performance)

C = 49

+5.74 1041.57 ± 19.61 min

Non-linearity hurts

RECURRING PATTERN

Combinations underperform individuals

Step + Temp, Opt C + Temp, and triple-combo all beat the baseline, but none beats Temp-MLP alone.

Scaling logits with a learned temperature is enough. Adding more nonlinearity (Option A: Post-Attention) or stacking mechanisms removes the gain. Light, targeted modifications outperform heavy ones.

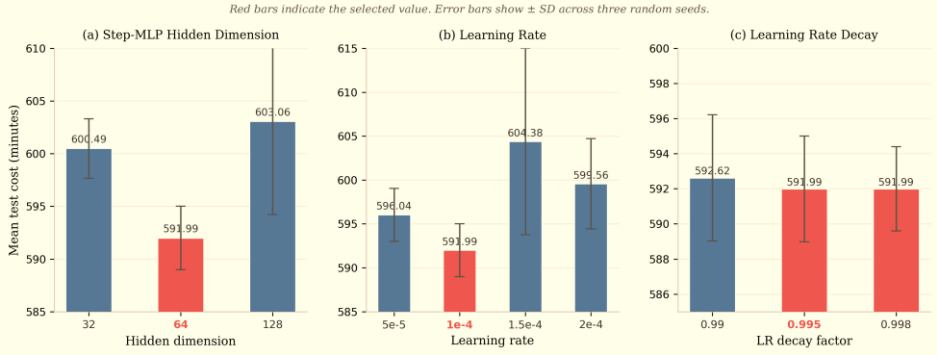
Training Stability and Hyperparameter Optimisation

Training Stability and Batch Size

Model	Batch	Mean ± SD (min)	Best Epoch
Baseline	32	1086.67 ± 13.84	9
Baseline	256	1095.93 ± 73.34	86
Step-MLP v2_light	32	1072.74 ± 43.31	35
Step-MLP v2_light	256	1068.77 ± 22.56	97

- Batch size was reduced (default:512) due to CUDA memory limitations at C=49.
- Batch size 32 produced early best epochs, indicating unstable training.
- Batch size 256 achieved more stable convergence and was selected for all remaining experiments.

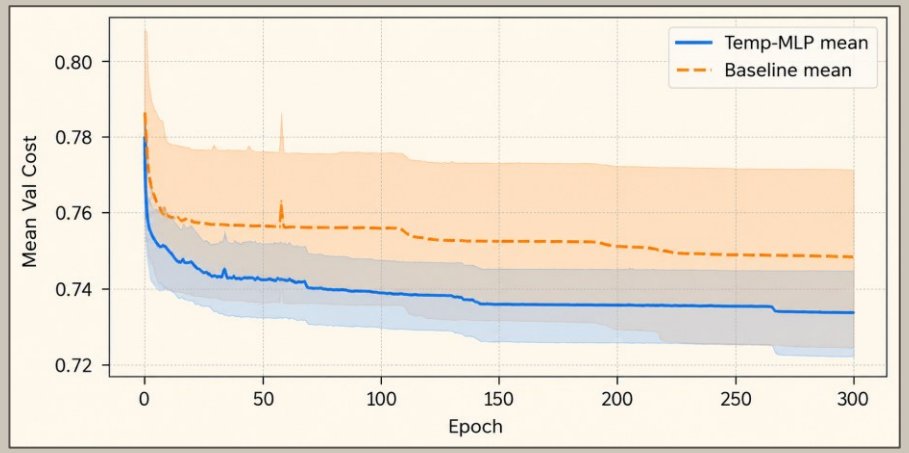
Hyperparameter Optimisation



- Hyperparameters were tuned sequentially to avoid expensive grid search.
- Hidden dimension 64 achieved the best performance.
- Learning rate 1×10^{-4} and decay 0.995 provided the most stable results.
- Epochs: 50 (C = 19/20), 100 (C = 29/49)
- Randon seeds: 1234, 5678, 9012

Validation Cost Trends Under Extended Training

Main experiments used 100 epochs. Extended to 300 epochs (3× compute budget) across all seeds to validate long-run behaviour.



1 Gap opens early

From epochs 20-30, the gap opens and remains stable for the rest of training.

2 Stable, not converged

Both curves still slowly decline at epoch 300, but the rate of improvement clearly diminishes. Additional training would provide only marginal gains. The relative ordering is unlikely to change.

3 Lower across-seed variance

Temp-MLP shows more consistent training behaviour across different random initialisations (narrower shared bands).

Cost Aware Gating

- Blend a classical heuristic logit h_i into the attention logits: $\ell_i^{\text{blended}} = \ell_i + \lambda_{\text{heur}} \cdot h_i(s_t)$
- We experimented with different λ values and found the best performance within $\lambda \in [0.25, 0.75]$, while larger values ($\lambda > 1.0$) reduced performance by over-weighting the heuristic.
- Therefore, $\lambda = 0.25$ was selected and applied to all heuristic methods due to computational limitations, although ideally each heuristic should be tuned with its own optimal λ.

C = 19

Fails to beat baseline

Baseline (100 epochs) $\mathbf{565.50 \pm 0.58}$

Fixed λ

NN (λ=0.25) 566.80 ± 0.58

NNR (λ=0.25) 572.90 ± 3.70

NN+1 (λ=0.25) 570.21 ± 3.24

---

fixed λ > learnable λ (linear: 575.43) > nonlinear transforms (piecewise: 575.43 and exponential: 567.83)

C = 49

3.67% improvement

Baseline $\mathbf{1095.93 \pm 73.34}$

Cost-aware (NN, λ=0.25) $\mathbf{1055.75 \pm 25.14}$ **(–3.67%)**

Opt C + Cost-aware $\mathbf{1068.17 \pm 26.05}$

Temp-MLP + Cost-aware $\mathbf{1062.78 \pm 35.90}$

---

standalone cost-aware > any combination.

Cost-aware gating is particularly beneficial for larger problem sizes, where the model struggles more and cost information provides additional guidance to improve performance.

Time slicing with safe refresh

- Replace the full-day traffic profile with a forward window of (W) bins and refresh the encoder as the decision time approaches the window edge.
- The experiments evaluate three configuration choices: window size (W = 3,6 and 9), refresh strategy, and start time(0,3,6 and 9).

Five hypotheses

H1 — **Shorter windows improve quality**
Mostly rejected. W=9 outperforms narrow windows in 2 out of 3 graph sizes (C=19, 20 and 29). Feeding the encoder with current and all remaining future data is more beneficial than a tight window.

H2 — **Frequent refresh improves performance**
Mostly confirmed. One-time refresh was selected as the practical choice for its simplicity, but periodic refresh wins in 3 out of 4 cases. The exception is C=29.

H3 — **Optimal config depends on graph size**
Confirmed. Window sizes, refresh strategies, and start bins all vary with graph size. Time slicing must be tuned to the problem scale, and no universal configuration exists.

H4 — **Start time impacts quality**
Confirmed with caution. Start bin 3 performs best, though this may be influenced by experimental setup. A full grid search would be needed to confirm the truly optimal configuration. Nevertheless, start time bin has a noticeable impact on performance.

H5 — **Time slicing reduces computational cost**
Rejected. Time slicing adds overhead but improves solution quality. It should therefore be viewed as a quality enhancer rather than a computational shortcut.

BEST RESULT AT C = 49

-1.19% solution-quality gain

W = 3, start bin = 3, periodic refresh (0.25)

Best configuration per graph size

Size	Window	Refresh	Δ Cost	Time
C = 19	W = 9	Periodic 0.25	-0.38%	+2.5%
C = 20	W = 3	One-time	-0.91%	+1.7%
C = 29	W = 9	One-time	-0.43%	+3.8%
C = 49	W = 3 **	Periodic 0.25	-1.19%	+10.7%

- Combining Temp-MLP with the best time slicing configuration achieved a 2.94% improvement over the baseline.
- However, the combined model did not outperform Temp-MLP alone (1063.75 min vs 1041.57 min), consistent with earlier observations that individual components often perform better than their combinations.
- The combination produced the lowest variance among all tested configurations (±6.47 min), indicating more stable solutions.

Time slicing is a quality enhancer, not a computational shortcut, trading 10% additional training time for cleaner inputs and better solutions.

Graph-Size-Independent Architecture

A key limitation of the baseline M1 model is its dependence on fixed customer set sizes. To support variable-sized inputs, adaptive average pooling is applied before each projection layer, producing a fixed 128-dimensional representation.

Model	Train	Test	Mean ± SD (min)	Δ vs. Baseline
Within-size				
Baseline	19	19	565.50 ± 0.58	–
Pooling	19	19	570.51 ± 2.99	+0.89%
Baseline	29	29	766.79 ± 7.39	–
Pooling	29	29	768.12 ± 7.87	+0.17%
Cross-size				
Pooling	19	29	770.15 ± 10.54	+0.44%
Pooling	29	19	572.52 ± 1.57	+1.24%

WITHIN-SIZE

The model is trained and tested on the same graph size.

+0.17% to +0.89% vs. size-specific baseline

Pooling degrades quality by only 0.89% at C = 19 and 0.17% at C = 29.

CROSS-SIZE

The model is trained on one graph size and tested on a different graph size.

≤ 1.24% degradation, no retraining

A single trained model serves any graph size
Deployment-friendly.

Comparison with Heuristic Baselines

Same 1,000 test instances, same cubic-spline travel times. Neural models are 3-seed averages.

	C = 19		*C = 49*	
Model / Heuristic	**Mean ± SD (min)**	**Time per Instance**	**Mean ± SD (min)**	**Time per Instance**
Neural Models				
Baseline (M1)	574.73 ± 10.78	**0.24 ms**	1095.93 ± 73.34	**0.64 ms**
Temp-MLP	568.68 ± 2.12	0.36 ms	1041.57 ± 19.61	0.94 ms
Cost-Aware (NN, λ = 0.25)	-	-	1055.75 ± 25.14	6.05 ms
Time Slicing (W = 9, start = 3)	572.55 ± 2.86	0.25 ms	1082.81 ± 26.19	0.65 ms
Classical Heuristics				
2-opt	575.65 ± 89.46	7,351.47 ms	1117.61 ± 91.79	91,349.73 ms
Nearest Neighbour (NN)	630.54 ± 105.49	9.80 ms	1154.21 ± 86.93	27.59 ms
Greedy Edge	633.71 ± 105.58	10.58 ms	1168.03 ± 100.46	23.83 ms
NNR	636.38 ± 101.28	94.58 ms	1201.84 ± 96.65	630.93 ms
NN+1	648.86 ± 104.86	110.21 ms	1211.52 ± 93.24	250.65 ms

- Neural models consistently outperform classical heuristics in both solution quality and inference speed across different graph sizes.
- Temp-MLP achieves the best overall performance, outperforming 2-opt by 1.2% at C=19 and 6.8% at C=49 while remaining significantly faster.
- The performance advantage increases with graph size, demonstrating the scalability of the neural approaches.

Recommendations

SMALL PROBLEMS (C = 19 / 20)

Step-MLP + Decoder Variants

- ❑ **RECOMMENDED** : Temp-MLP standalone and it reduces cost by 6 min
- ❑ All decoder variants outperform the baseline.
- ❑ Combining decoder variants degrades performance.

Cost-Aware Gating

- ❑ None of the tested configurations outperformed the baseline.
- ❑ Fixed lambda outperforms learnable lambda.

Time Slicing with Safe Refresh

- ❑ **The best configuration** : W = 3 with one-time refresh at C = 20, and W = 9 with periodic refresh (interval = 0.25) at C = 19
- ❑ Adds training overhead (approximately 2.5% at C = 19)
- ❑ Improves solution quality by 0.38%

LARGER PROBLEMS (C = 49)

Step-MLP + Decoder Variants

- ❑ **RECOMMENDED** : Temp-MLP standalone and it achieves a 4.96% improvement.
- ❑ Post-attention glimpse refinement (Option A) degrades performance, suggesting that the original linear projection is more effective.
- ❑ Combining decoder variants degrades performance, indicating that individual components are more effective than their combinations.

Cost-Aware Gating

- ❑ Standalone cost-aware gating ($\lambda = 0.25$) with a linear NN heuristic achieved a 3.67% performance improvement.
- ❑ Combining cost-aware gating with other components degraded performance in all tested combinations.

Time Slicing with Safe Refresh

- ❑ **The best configuration** : W = 3 with periodic refresh (interval = 0.25)
- ❑ Adds training overhead (approximately 10.7%)
- ❑ Improves solution quality by 1.19%

Overall

- ❑ **Adaptive pooling** mitigates cross-size generalisation issues while limiting quality degradation to less than 1.24%.
- ❑ Neural models outperform all heuristics at C = 19 and C = 49. 2-opt is the strongest heuristic but has much higher inference time.
- ❑ Temp-MLP is the recommended extension for all graph sizes.
- ❑ Individual components may show better performance, but combining them does not guarantee further gains.

Limitations

DATA

Single dataset

All experiments used one real-world dataset (Beijing). Cross-dataset generalisation untested.

PROBLEM SIZE

Capped at C = 49

Baseline (Zhang et al., 2023) tested up to C=19; this work extends to C=49. Larger sizes remain unexplored.

COMPUTE

100 epochs vs. baseline's 1,000

Most runs at 100 epochs due to budget. 300-epoch runs confirm ranking is stable, but both models likely undertrained. Further gains expected with more compute.

HYPERPARAMETERS

Sequential search, not grid / Bayesian

Better configurations may exist outside the explored regions.

TEMP-MLP RANGE

τ fixed to [0.5, 2.5]

The best-performing configuration (Temp-MLP) scales attention logits with $\tau \in [0.5, 2.5]$. Wider or asymmetric ranges (e.g. [0.1, 5.0]) were not explored and may further improve the strongest configuration.

Future Work

DEEPER TEMP-MLP STUDY

Investigate Temp-MLP scaling behaviour

Most consistent gains across sizes; behaviour and scaling factor τ warrant dedicated analysis. The Temp-MLP + Time Slicing combo (lowest variance) deserves longer-run study to test whether stability eventually translates to quality.

SCALE & DATA

Larger graphs and more datasets

Neural advantage over heuristics grew between C=19 and C=49. Test larger sizes ($C \geq 100$). Replace the fixed Beijing dataset with synthetic generators or other cities to develop a more generalised model.

PROBLEM CLASS

Stochastic travel times

Extend the deterministic TDTSP framework to stochastic edge weights. This is more realistic for live traffic and a natural next step.

ADAPTIVE TIME WINDOW

Self-adjusting window size

Current window is fixed. Optimal size depends on tour progress: wider early, narrower late. An adaptive scheme that shrinks the window as customers are served should improve solution quality further.

TRAINING FRAMEWORK

Actor-critic with the new modifications

Zhang et al. (2023) tried both REINFORCE+rollout and actor-critic. This work used REINFORCE. Pairing actor-critic with Temp-MLP, cost-aware gating, and time slicing is unexplored.

Conclusion

- Extended the M1 TDTSP model of Zhang et al. (2023) for dynamic routing with real-world traffic data.
- Proposed three architectural extensions:
  - Step-aware decoder (Step-MLP / Temp-MLP)
  - Cost-aware gating
  - Time slicing with safe refresh
- Temp-MLP was the most effective decoder enhancement, improving solution quality by 1.05% (C=19) and 4.96% (C=49) over the baseline.
- v2_light with linear time encoding was the best feature set; linear time was the most influential feature.
- Cost-aware gating improved performance at C=49 (+3.67%), but not at C=19.
- Time slicing with safe refresh improved solution quality at both graph sizes, despite additional training overhead.
- Combining individual components did not consistently improve performance over the best standalone methods.
- Adaptive pooling enabled graph-size-independent inference with less than 1.24% quality degradation.
- Neural models outperformed all classical heuristics, while maintaining significantly faster inference than 2-opt.

References

Bello, I., Pham, H., Le, Q. V., Norouzi, M., and Bengio, S. (2017). Neural combinatorial optimization with reinforcement learning. In *International Conference on Learning Representations Workshop*.

Chen, X. et al. (2025). Dynamic traveling salesman problem with time-dependent stochastic travel times. *Transportation Research Part C: Emerging Technologies*.

Choo, J., Kwon, Y.-D., Kim, J., Jae, J., Hottung, A., Tierney, K., and Gwon, Y. (2022). Simulation-guided beam search for neural combinatorial optimization. In *Advances in Neural Information Processing Systems*.

Cordeau, J.-F., Ghiani, G., and Guerriero, E. (2014). Analysis and branch-and-cut algorithm for the time-dependent travelling salesman problem. *Transportation Science*, 48(1):46–58.

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812.

Dai, H., Khalil, E. B., Zhang, Y., Dilkina, B., and Song, L. (2017). Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems*, volume 30.

Drakulic, D., Michel, S., Mai, F., Sors, A., and Andreoli, J.-M. (2023). BQ-NCO: Bisimulation quotienting for efficient neural combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 36.

Haghani, A. and Jung, S. (2005). A dynamic vehicle routing problem with time-dependent travel times. *Computers & Operations Research*, 32(11):2959–2986.

He, K., Zhang, X., Ren, S., and Sun, J. (2015). Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(9):1904–1916.

Ichoua, S., Gendreau, M., and Potvin, J.-Y. (2003). Vehicle dispatching with time-dependent travel times. *European Journal of Operational Research*, 144(2):379–396.

Joshi, C. K., Laurent, T., and Bresson, X. (2019). An efficient graph convolutional network technique for the travelling salesman problem. *arXiv preprint arXiv:1906.01227*.

Kool, W., van Hoof, H., and Welling, M. (2019). Attention, learn to solve routing problems! In *International Conference on Learning Representations*.

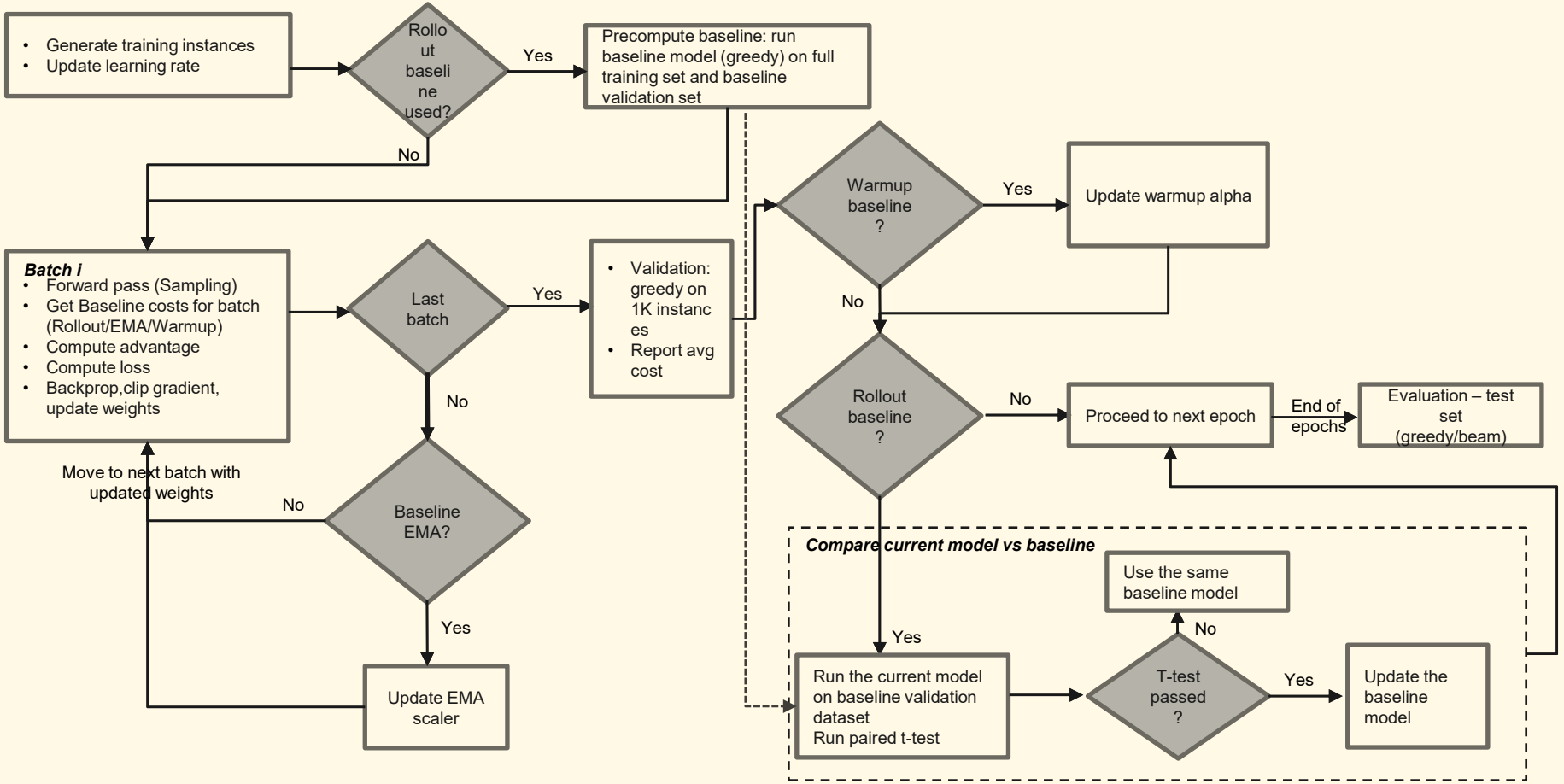
Malandraki, C. and Daskin, M. S. (1992). Time dependent vehicle routing problems: Formulations, properties and heuristic algorithms. *Transportation Science*, 26(3):185–200.

Malandraki, C. and Dial, R. B. (1996). A restricted dynamic programming heuristic algorithm for the time dependent traveling salesman problem. *European Journal of Operational Research*, 90(1):45–55.

Thank You!

Do you have any questions?

Training Process and Baseline Update



Training Process and Baseline Update

Each training epoch follows this structure:

1. The learning rate is decayed by multiplying with a scalar (default:0.995).
2. A new training dataset is generated by random sampling (default:128,000 instances) to reduce overfitting.
3. If using the rollout baseline, the frozen baseline model performs greedy decoding on the entire training dataset at the start of the epoch. The resulting costs b_i are stored and paired with each training instance throughout the epoch.
4. The dataset is divided into mini-batches (default batch size: 512). Each batch follows the per-batch training step described in the right-hand side.
5. After all batches, the current model is evaluated on the fixed validation dataset (1,000 instances) using greedy decoding, and the mean validation cost is reported.
6. If using the warmup baseline, the blend factor $\gamma(e)$ is updated
7. If using the rollout baseline, the baseline model is compared with the current model via a paired t-test on the baseline validation dataset. The baseline is updated only if the current model shows statistically significant improvement.
8. Model state, optimiser state, and baseline state are saved at specified intervals.

Per-Batch Training Step

1. Sample tours using the training model (stochastic sampling mode)
2. Compute: Tour cost C Log-likelihood logp θ ($\pi|x$)
3. Compute the baseline value (retrieved)
4. Calculate the REINFORCE loss according to the selected strategy
5. Perform backpropagation.
6. Apply gradient clipping
7. Update model parameters.
8. After training, evaluate the best model on the test set using greedy/beam search decoding.